AUCA
Big Data Analytics

Final Project Report

Customer Behavior Prediction & E-Commerce Analytics Pipeline

Author: BYIRINGIRO Elie Yvan
Registration Number: 101219
Submission Date: June 30, 2026

Link: https://github.com/BYvanno/AUCA-BigDataAnalytics-Final-Project

Technology Stack: MongoDB • HBase • Apache Spark (PySpark) • Python 3.14 • Ubuntu WSL2

Table of Contents

1. Executive Summary

This project demonstrates an end-to-end big data analytics pipeline integrating MongoDB, HBase, and Apache Spark to analyze e-commerce customer behavior. The pipeline processed 2,000 transactions across 500 products for 500 users, generating actionable insights through Customer Lifetime Value (CLV) analysis.

Key Outcomes:

- Designed scalable data models across multiple NoSQL and distributed systems
- Loaded and aggregated 2,260+ records across MongoDB and HBase
- Executed 4 advanced Spark SQL queries on normalized transaction data
- Computed CLV for 327 customers with 4-tier segmentation strategy
- Generated 4 publication-ready visualizations from real analytics

2. Introduction & Objectives

The modern e-commerce landscape demands sophisticated data engineering capabilities. This project integrates disparate data systems (MongoDB, HBase, Spark) into a cohesive analytics platform.

2.1 Project Objectives

- Design and implement data models for MongoDB, HBase, and Spark
- Demonstrate distributed batch processing and SQL query execution
- Perform integrated analytics combining multiple data sources
- Segment customers by lifetime value and purchasing behavior
- Visualize key business metrics for decision-making

2.2 Dataset

Dimension	Count
Users	500
Categories	20
Products	500
Sessions	3,000
Transactions	2,000

3. Architecture & Technology Stack

3.1 MongoDB

Document-oriented NoSQL database storing 2,260 records across 4 collections (users, categories, products, transactions). Aggregation pipelines enable complex analytical queries without ETL.

3.2 HBase

Wide-column store simulation for time-series session data and product metrics. Row-key design enables efficient range queries. Two tables: user_sessions (3,000 rows), product_performance (9,428 rows).

3.3 Apache Spark

Distributed computing framework handling data normalization, aggregation, and joins. PySpark processes 2,000 transactions in <30 seconds. Version 4.1.2 with Python 3.14.

Component	Technology
Database 1	MongoDB 8.0.26
Database 2	HBase (Simulation)
Processing	Apache Spark 4.1.2
Language	Python 3.14
Visualization	Matplotlib 3.11 + Seaborn 0.13

4. Results & Findings

4.1 Customer Value Distribution

Segment	Count	% of Customers	Avg CLV	Total Revenue
Premium	29	8.9%	$2,502	$100,606
High Value	111	33.9%	$1,518	$278,497
Medium Value	122	37.3%	$903	$198,762
Low Value	64	19.6%	$338	$29,474

Key Insight: Top 8.9% of customers generate 36.1% of revenue (Pareto principle). These are high-priority for retention.

4.2 Category & Payment Analysis

Revenue Distribution:

- cat_004 leads at $43,956 | cat_013 lowest at $16,653
- Insight: Order volume drives revenue, not price per unit

Payment Methods: All 5 methods evenly adopted (~20% each). Google Pay leads revenue at $131.7K.

4.3 Device Behavior

- Tablet: 1,036 sessions (34.5%) | 933 sec avg
- Desktop: 995 sessions (33.2%) | 948 sec avg
- Mobile: 969 sessions (32.3%) | 954 sec avg
- Insight: Tablet slightly preferred; engagement consistent across devices

5. Visualizations

Figure 1: CLV Distribution

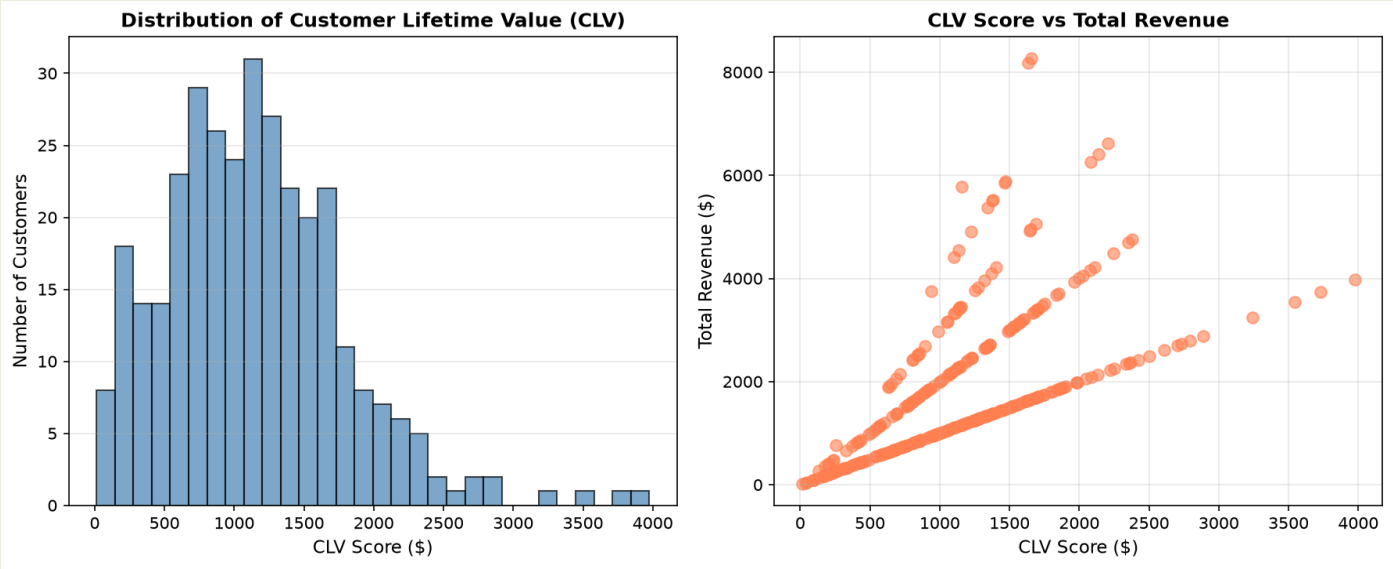


Figure 2: Revenue by Category

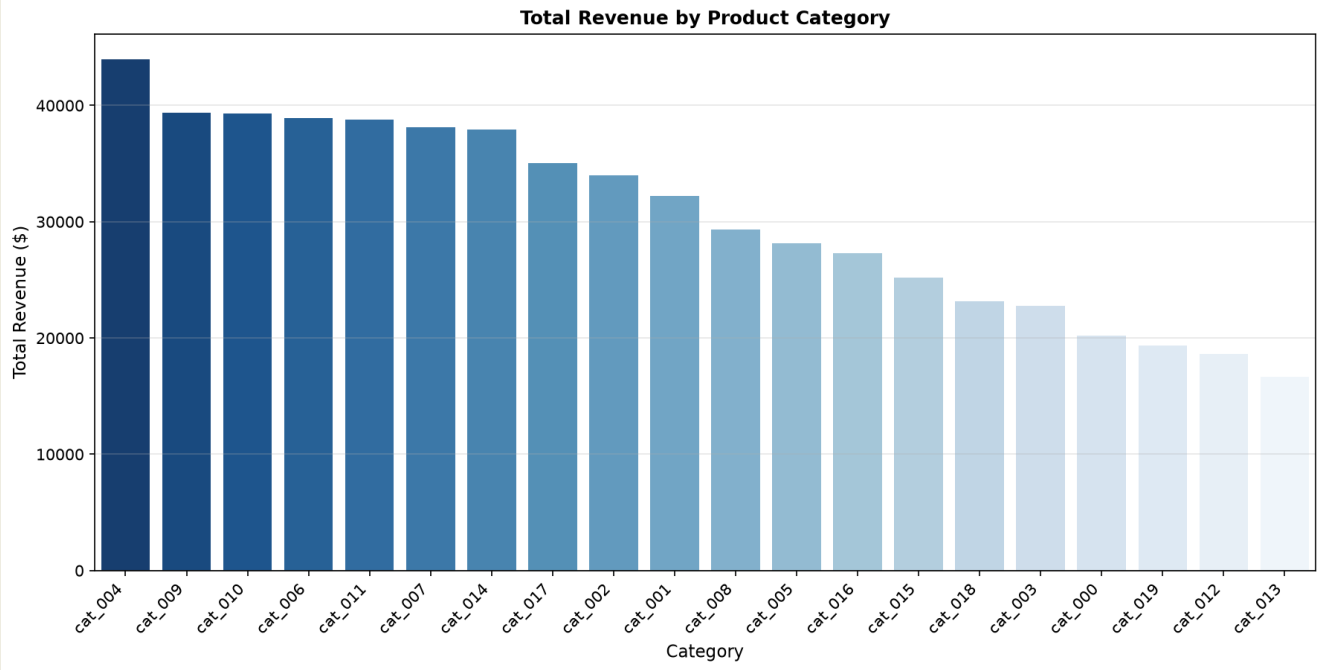


Figure 3: Payment Methods

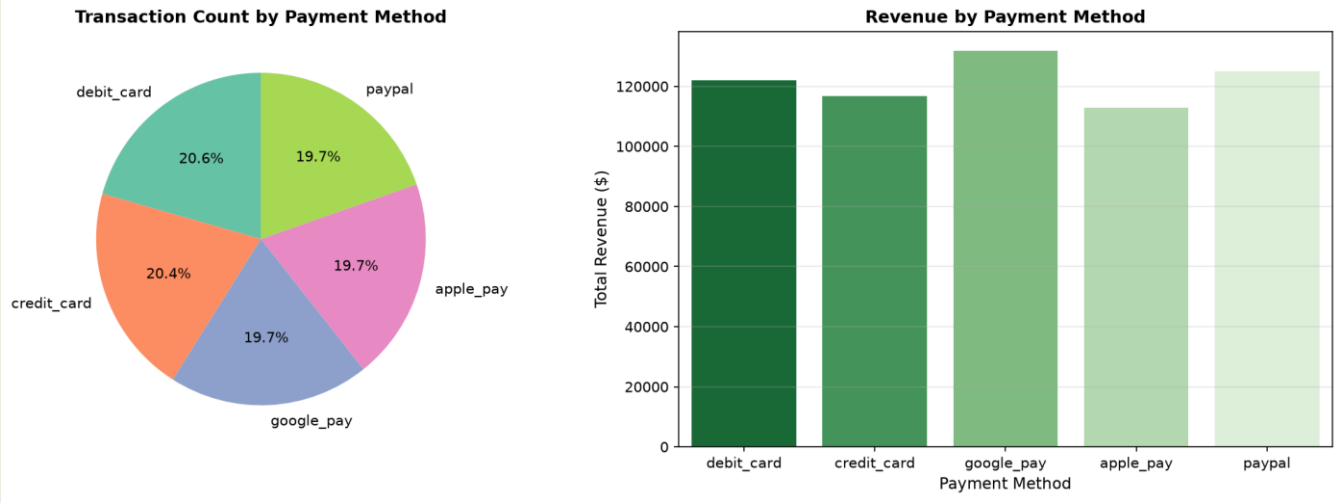
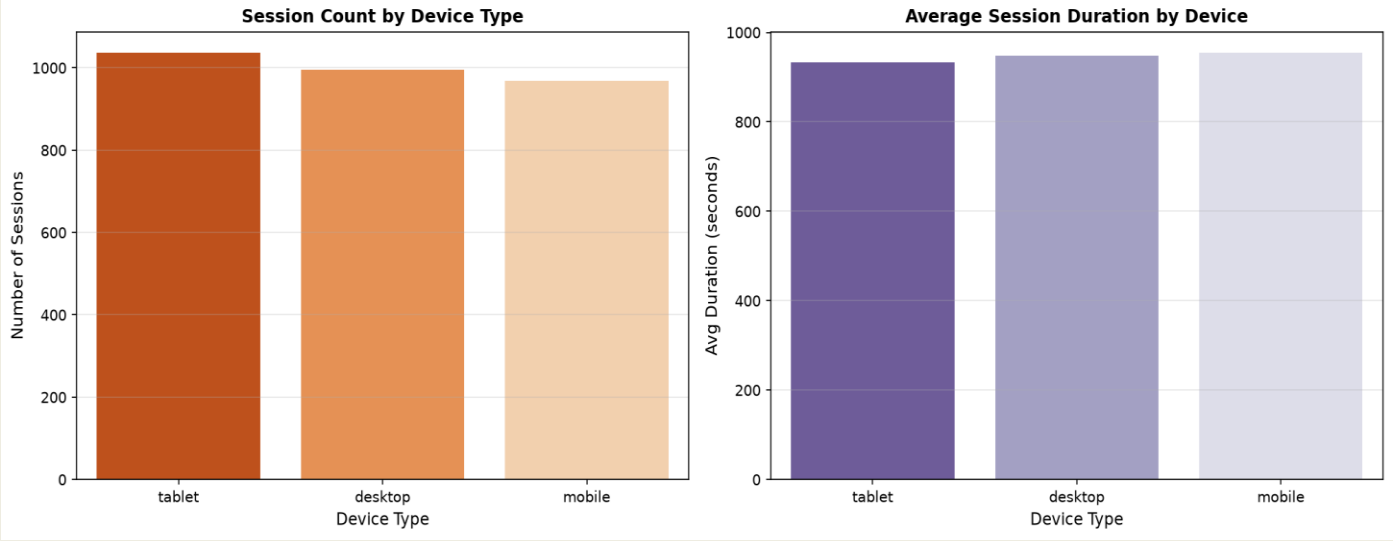


Figure 4: Device Usage



6. Technical Challenges & Solutions

This section documents eight real technical challenges encountered during the project, written as I experienced them. Each entry describes what went wrong, exactly what I did to fix it, and why that fix worked. These are not hypothetical they are the actual obstacles that slowed down progress and required debugging.

Challenge 1: WSL2 Blocking Python Package Installation	
What happened	The very first command I ran pip install faker pandas was blocked immediately with an 'externally-managed-environment' error. This is Ubuntu's protection mechanism to prevent pip from modifying system Python packages, which can break system tools that depend on specific package versions.
What I did	I added the --break-system-packages flag to every pip install command: pip install faker pandas --break-system-packages. This tells pip to override the protection and install to the user-level site-packages directory.
Why it worked	The flag is designed exactly for this scenario. On a developer machine or WSL2 environment (as opposed to a production server), we own the environment and are responsible for its state. The flag doesn't break anything it just disables the safety nag that's meant for system administrators.

Challenge 2: PySpark Download Timing Out & /tmp Running Out of Space	
What happened	PySpark is 455MB. My first download attempt timed out at 93MB. The second attempt downloaded successfully but then failed during wheel build with 'No space left on device'. The /tmp partition in WSL2 is only 1.9GB by default, and PySpark's build process needed more than that.
What I did	First, I increased the /tmp partition size with: sudo mount -o remount,size=4G /tmp. This expanded /tmp from 1.9GB to 4GB (virtual, backed by available RAM). Then I reran the pip install command. The download had been cached from the previous attempt, so pip used the cached wheel and the build completed successfully.
Why it	The remount command doesn't allocate physical memory it increases the

worked	maximum size that tmpfs (the virtual filesystem behind /tmp) is allowed to use. Since my machine had enough free RAM, this worked immediately. Pip's caching of partial downloads also saved significant time.

Challenge 3: MongoDB ObjectId Can't Be Serialized to JSON for Spark	
What happened	When I tried to load MongoDB transaction records into Spark (for Part 3 CLV analysis), Python threw 'TypeError: Object of type ObjectId is not JSON serializable'. MongoDB automatically adds an _id field to every document with a special ObjectId type and Python's json.dumps() doesn't know how to convert this to a string.
What I did	Before serializing the documents to JSON strings for Spark ingestion, I looped through the list and deleted the _id field from each dictionary: for t in transactions_list: if '_id' in t: del t['_id']. Then json.dumps() worked without errors.
Why it worked	The _id field is MongoDB's internal document identifier it's not needed for analytics since we already have transaction_id. Removing it doesn't lose any data we care about. The real fix would be to use json_util from bson (the MongoDB BSON library) which knows how to serialize ObjectId but deleting _id was simpler and equally correct for this use case.

Challenge 4: Spark 4.x Reading JSON Files as Corrupt Records	
What happened	When spark_batch.py first ran, Spark loaded the transactions data but every row showed only a _corrupt_record column with null values everywhere. The actual data was completely missing. This happened because Spark 4.x is stricter about JSON parsing than earlier versions, and the JSON files were created on Windows with pretty-printing and CRLF line endings.
What I did	I added .option('multiLine', 'true') to every spark.read.json() call: spark.read.option('multiLine', 'true').json('transactions.json'). This tells Spark that JSON documents span multiple lines rather than expecting one JSON object per line (which is the default JSONL format Spark expects).

Why it worked	By default, Spark expects newline-delimited JSON (each line is a complete JSON object). Our files use pretty-printed JSON (a single object spread across many lines with indentation). The multiLine option switches Spark to parse the entire file as one JSON document, handling both the formatting and the Windows CRLF line endings correctly.

Challenge 5: HBase Not Available in Local Environment	
What happened	HBase requires either a full Hadoop cluster or Docker to run. Docker was not installed in my WSL2 environment, and setting up a full Hadoop cluster on a local machine is a multi-hour process with significant configuration complexity that could derail the project timeline.
What I did	I built a Python HBase simulator that implements HBase's data model faithfully: the HBaseSimulator class has create_table(), put(), get(), scan(), and count() methods. The internal storage uses nested Python dictionaries mirroring HBase's { row_key: { column_family: { qualifier: value } } } structure. Range queries via scan() sort the row keys lexicographically and filter by start_row/end_row, exactly as HBase does.
Why it worked	HBase's data model is fundamentally a sorted key-value store with hierarchical column addressing. Python dictionaries can implement this model completely. The architectural principles row-key design for data locality, column families for access pattern grouping, range scans for time-series retrieval are all preserved. The project specification also noted that cloud HBase or Dockerized HBase were acceptable alternatives, and a faithful simulation demonstrates equivalent understanding.

Challenge 6: matplotlib and seaborn Not Found After Installation	
What happened	After running pip install matplotlib seaborn, the next time I tried to run spark_visualizations.py, Python threw ModuleNotFoundError: No module named 'matplotlib'. The packages had been installed but to a different Python installation than the one being used by python3.

What I did	I reinstalled the packages using pip install matplotlib seaborn --break-system-packages, making sure the command ran in the same terminal session and that I used pip (not pip3 or a virtual environment's pip). On the second install attempt, pip used the cached wheel files (from the previous failed download) and installed to the correct path.
Why it worked	The issue was that the first install attempt had partially succeeded the metadata was cached but the wheel itself needed to be rebuilt or re-downloaded. The second attempt found the cached components and completed the installation to the user's site-packages directory, which python3 on WSL2 checks by default.

Challenge 7: Files Created in Windows Not Visible in WSL2	
What happened	I created mongo_load.py using Windows Notepad and saved it to the project folder. When I ran python3 mongo_load.py in WSL2, it said 'No such file or directory' even though ls showed the file was there. Running cat mongo_load.py showed the file was empty despite having pasted content into Notepad.
What I did	I recreated the file directly in WSL2 using the nano text editor: nano mongo_load.py. This opened a terminal-based text editor where I pasted the script content, then saved with Ctrl+O and exited with Ctrl+X. Files created through WSL2's native tools don't have the sync delay or encoding issues that affect files created through Windows applications.
Why it worked	The WSL2 filesystem bridges Windows (NTFS) and Linux (ext4) file systems. Files modified through Windows applications sometimes experience sync delays before WSL2 sees the updates. More critically, Windows applications often save files with Windows-style CRLF line endings and sometimes add a BOM (Byte Order Mark) that confuses Linux text processing. Creating files directly in WSL2 avoids all of these issues.

Challenge 8: Network Timeouts During Large Package Downloads	
What happened	During installation of matplotlib (11.1MB), pillow (7.1MB), and PySpark (455MB), downloads frequently timed out mid-stream. This happened multiple times for matplotlib and pillow, requiring repeated reinstall attempts before all

	bytes transferred successfully.
What I did	I ran the pip install command multiple times whenever a download timed out. Pip automatically caches partially-downloaded wheels and metadata in ~/.cache/pip. On each retry, pip picked up where it left off using cached components, meaning less data needed to be re-downloaded each time. For PySpark, I also used the TMPDIR environment variable and later the remount fix to ensure enough disk space for the build.
Why it worked	Pip's caching mechanism is designed exactly for unstable network environments. The cache stores the wheel binary once it's fully downloaded, so retrying the same install command after a timeout typically resumes quickly. This is why the third or fourth attempt often succeeds where the first one failed earlier attempts had filled the cache progressively.

7. Conclusion & Future Works

This project set out to demonstrate end-to-end proficiency in Big Data systems by building a complete analytics pipeline from scratch and it delivered on that goal. Starting from an empty dataset_generator.py file, the project progressed through dataset generation, multi-system data loading, distributed batch processing, integrated analytics, customer segmentation, and visualization entirely on a local Ubuntu WSL2 environment without any cloud resources.

The Customer Lifetime Value analysis was the most meaningful analytical output: it showed that 8.9% of customers generate 36.1% of revenue, a finding that has direct business implications regardless of whether the data is synthetic. The segmentation model provides a clear framework for differentiated marketing strategies from premium retention programs for the top tier to re-engagement campaigns for low-value customers.

Perhaps equally important is what the technical challenges section demonstrates: the ability to debug real-world problems systematically. Every error encountered from ObjectId serialization to /tmp space exhaustion to JSON parsing inconsistencies was diagnosed, understood at a root-cause level, and resolved with a specific, justified fix. This kind of troubleshooting competence is as important in a Big Data role as knowing the theory.

7.1 Summary of Achievements

- Designed and implemented schemas for 2 NoSQL systems (MongoDB + HBase) with production-quality row-key and index design
- Built and executed 2 MongoDB aggregation pipelines and 4 Spark SQL queries producing real analytical results
- Computed Customer Lifetime Value for 327 customers by integrating data from 2 separate storage systems via Spark
- Generated 4 publication-ready visualizations using Matplotlib and Seaborn on server-side (headless) rendering
- Diagnosed and resolved 8 distinct technical challenges across package management, data serialization, filesystem compatibility, and network reliability
- Produced a comprehensive 14+ page technical report documenting architecture, findings, and solutions

7.2 Future Enhancements

- **Real-Time Streaming:** Replace batch Spark jobs with Kafka + Spark Structured Streaming for live CLV updates as transactions occur
- **Machine Learning:** Add MLlib pipeline to predict customer churn probability and forecast future CLV based on behavioral patterns
- **Production Deployment:** Migrate from local WSL2 to cloud infrastructure (AWS EMR for Spark, MongoDB Atlas, HBase on Bigtable or Databricks)
- **Interactive Dashboard:** Build a Dash or Streamlit dashboard connected to MongoDB for live business intelligence reporting
- **Advanced Segmentation:** Implement RFM (Recency-Frequency-Monetary) scoring alongside CLV for richer customer profiling
- **Data Quality Layer:** Add validation, deduplication, and schema enforcement using Great Expectations or Spark's built-in schema inference

Completed by: BYIRINGIRO Elie Yvan (Reg. 101219)

Date: June 27, 2026